

Product Requirements Document

Geometry Agent Harness

ForgeCAD · AI-Powered CAD Agent Platform

Field	Value
Document Type	Product Requirements Document
Source	Engineering Design Review — Architecture Decision Memo
Date	May 2026
Audience	Engineering Leads
Decision	Approve Direction + Surface Risks
Core Stack	RLM + Temporal + CadQuery + MeshLib + ForgeCAD
Research Baseline	github.com/jabarkle/CADSmith (cloned)
Status	Pending Approval

Table of Contents

1. [Executive Recommendation](#)
2. [Business Context and Platform Goals](#)
3. [System Architecture](#)
4. [Spatial Reasoning Requirements — Thinking in 3D](#)
5. [Canonical State Ownership](#)
6. [Geometry Agent Runtime Contract](#)
7. [Minimum Viable Workflow](#)
8. [Worker Boundaries and Execution Surfaces](#)
9. [State Management and Durability](#)
10. [Product API Contract](#)
11. [Performance Requirements and Decision Metrics](#)
12. [Implementation Roadmap](#)
13. [Risks and Mitigations](#)
14. [Trace Capture and Evaluation Pipeline](#)

0. Repository Baseline — CADSmith

0.1 What CADSmith Is

The repository at (<https://github.com/jabarkle/CADSmith>) (cloned into this project) is a research-grade multi-agent system that generates manufacturing-ready CAD models from natural language. Developed at Carnegie Mellon University's Department of Mechanical Engineering (Barkley, Loghmani, Faramani) and formerly known as AutoFab, it combines LLM code generation with programmatic geometric validation from the OpenCASCADE (OCCT) kernel and visual inspection from an independent vision-language model, iterating until the part is dimensionally correct — not just visually plausible.

CADSmith establishes the core loop this platform productizes. It explicitly rejects screenshot-only verification: parts that *look* right but carry a 3 mm bounding box error are useless for manufacturing. This constraint is inherited by the production platform as a hard requirement.

0.2 Repository Structure

```
CADSmith/
  autofab/
    pipeline.py      # Main orchestration: Planner → Coder → Executor → Validator →
Refiner
    agents.py        # LLM agent definitions: Planner, Coder, Error Refiner, Judge,
Refiner
    executor.py      # Sandboxed subprocess execution, OCCT geometry extraction,
STEP/STL export
    validator.py     # Solid validity check + LLM-as-Judge with optional vision
    render.py        # VTK three-view rendering (isometric, high-angle rear, front
profile)
    metrics.py       # Chamfer Distance, F1 score, Volumetric IoU against reference
STLs
    rag_kb1.py       # RAG KB: 155 CadQuery API entries + 28 worked examples
    rag_kb2.py       # RAG KB: 25 error-solution patterns for common CadQuery
failures

  data/dataset_v2/
    t1_primitives.jsonl      # 50 prompts: boxes, cylinders, cones, tori (1-3
ops)
    t2_engineering_parts.jsonl # 25 prompts: brackets, flanges, gears, plates (3-
8 ops)
    t3_complex_parts.jsonl   # 25 prompts: lofts, sweeps, shells, multi-body
(5-15 ops)
```

```
scripts/
  run_custom_benchmark.py      # Full pipeline benchmark across all 100 entries
  run_zeroshot_baseline.py     # Zero-shot baseline (single LLM call, no agents)
  analyze_results.py           # Load results, compute summaries, compare to
baselines

run.py                         # Single-prompt entry point
requirements.txt
```

0.3 CADSmith Benchmark Results (Reference Baseline)

All metrics computed in absolute millimeter space with ICP alignment.

Configuration	Exec %	CD Median	CD Mean	F1 Median	IoU Median
Zero-shot (single LLM call)	95%	0.55	28.37	0.9707	0.8085
Full pipeline, no vision	99%	0.48	18.19	0.9792	0.9563
Full pipeline with vision	100%	0.48	0.74	0.9846	0.9629

Key findings the platform must account for:

- Vision yields a **38× reduction in mean Chamfer Distance** versus zero-shot
- Removing vision from T3 complex parts raises mean CD from **1.42 to 49.68** — catastrophic degradation
- 100% execution rate is only achieved with the full vision-augmented pipeline
- Known near-miss failure (T3_019 drone frame): F1 = 0.963, IoU = 0.985 — passed all checks but contained small arm-hub gaps invisible to fixed three-view rendering. This establishes the known ceiling of fixed-view verification and must be catalogued as a failure taxonomy entry.

0.4 Layer Mapping

CADSmith File	Platform Layer	PRD Section
<code>pipeline.py</code>	Layer GRT — Runtime orchestration baseline	§3.2, §7
<code>agents.py</code> — Planner	Layer RLM — Primitive planning agent	§3.2, §6.2, §8.2
<code>agents.py</code> — Coder, Refiner	Layer GRT — CadQuery compilation + outer repair	§6.2, §7.2, §8.2

CADSmith File	Platform Layer	PRD Section
<code>agents.py</code> — Error Refiner	Layer GRT — Inner execution-error repair loop	§7.2, §8.2
<code>agents.py</code> — Judge (Claude Opus)	Layer GRT — Independent verifier	§4, §6.2, §8.2
<code>executor.py</code>	Layer CAD — OCCT execution + STEP/STL export	§3.2, §6.2
<code>validator.py</code>	Layer GRT — <code>visual_verify</code> + <code>mesh_inspect</code> primitives	§6.2
<code>render.py</code>	Layer GRT — <code>render_views</code> primitive	§4.4, §6.2
<code>metrics.py</code>	Evaluation harness — CD, F1, IoU	§11, §14
<code>rag_kb1.py</code>	Layer RLM — CadQuery API + example context	§3.2, §6.2, §6.5
<code>rag_kb2.py</code>	Layer RLM — Error-pattern repair context	§6.2, §6.5, §7.2
<code>data/dataset_v2/</code>	Trace/Eval — Benchmark prompt corpus	§14
<code>scripts/run_custom_benchmark.py</code>	Trace/Eval — Regression pipeline	§14.3
<code>scripts/run_zeroshot_baseline.py</code>	Trace/Eval — A/B zero-shot baseline	§11.2
<code>scripts/analyze_results.py</code>	Trace/Eval — Result schema prototype	§14

1. Executive Recommendation

1.1 Approval Condition

The architecture direction is approved with the condition that geometry reasoning is treated as a first-class runtime capability rather than a thin wrapper around an editor. Approving this direction commits the team to building a platform where CadQuery, MeshLib, and a Geometry Agent Runtime are genuine geometry authorities, not decorative additions to ForgeCAD.

1.2 Platform Scope

This document specifies requirements for the **Geometry Agent Harness**: a reusable platform capability for evaluating and building AI agents for CAD workflows. The harness productizes and extends the multi-agent loop proven in CADSmith (`pipeline.py`), adding durable

orchestration via Temporal, human-in-the-loop approval gates, ForgeCAD editable handoff, and a structured trace flywheel for evaluation and model improvement.

The architecture combines:

- **Recursive Language Models (RLM)** for context exploration and decomposition, building on CADSmith's Planner agent (`agents.py`) and RAG knowledge bases (`rag_kb1.py`, `rag_kb2.py`)
- **Temporal.io** for durable workflow orchestration — not present in CADSmith; a required addition for production
- A **Geometry Agent Runtime** whose core execution model derives from CADSmith's (`pipeline.py`) and (`executor.py`), extended with typed semantic primitives, MeshLib, and ForgeCAD handoff

1.3 Business Case Statement

The business case is not limited to faster part generation. The platform must create an auditable loop for CAD automation where:

1. Natural-language intent becomes typed geometric operations via semantic primitives
2. Every attempt produces structured traces persisted as artifacts
3. Visual and deterministic checks evaluate geometry quality — CADSmith's benchmark quantifies why both are necessary (§0.3)
4. Successful and failed traces become reusable evaluation and training data, seeded from `data/dataset_v2/`

1.4 Central Feasibility Requirement

CAD agents cannot rely on text-only planning or screenshot-only verification. CADSmith's benchmark data makes this quantitative: removing visual verification raises T3 mean Chamfer Distance from 1.42 to 49.68. The system must implement both **Thinking with Images** and **Thinking in 3D** as validated by CADSmith's Judge-plus-kernel approach (`validator.py` + `executor.py`).

Design Review Position: The review question is not whether an LLM can write CAD code once. The question is whether the system can observe, inspect, repair, and explain geometry across repeated attempts.

2. Business Context and Platform Goals

2.1 Platform Definition

The system shall implement a reusable harness for CAD agents that allows teams to create, evaluate, audit, and improve agents that design geometry from intent. ForgeCAD provides an editable code-first surface. The platform value derives from the full loop: planning, 3D reasoning,

deterministic inspection, durable execution, trace capture, and model improvement.

CADSmith's `pipeline.py` is the research proof-of-concept for this loop; this PRD specifies its production form.

2.2 Required Platform Capabilities

2.2.1 Agent Evaluation Platform

The system must support running repeatable CAD tasks, capturing structured traces, comparing model behavior, and scoring outputs against rubrics. CADSmith's

`scripts/run_custom_benchmark.py` and `metrics.py` (Chamfer Distance, F1, IoU) provide the evaluation kernel; the platform must integrate and extend these for production use.

2.2.2 Lower Iteration Cost

The system must automate routine design attempts while preserving human review for risky outputs. CADSmith's inner loop (up to 3 execution retries via Error Refiner + `rag_kb2.py`) and outer loop (up to 5 geometric refinement iterations via Refiner agent) establish the repair depth baseline.

2.2.3 Spatial Reasoning

The system must extend Thinking with Images into multi-view and geometry-aware Thinking in 3D. CADSmith's `render.py` (VTK three-view: isometric, high-angle rear, front profile) and the independent Judge in `validator.py` (Claude Opus) demonstrate the minimum viable visual reasoning approach.

2.2.4 Durable Workflows

The system must use Temporal.io to survive crashes, retries, approval waits, and long-running jobs. CADSmith has no durable orchestration layer; this is a complete production addition. No agent run may lose state due to infrastructure failure.

2.2.5 Editable Deliverables

The system must return ForgeCAD code and exports that engineering users can inspect and modify. CADSmith's `executor.py` already exports STEP and STL from the OCCT kernel; the platform extends this with ForgeCAD model code as the primary editable handoff surface.

2.2.6 Trace Flywheel

The system must convert successful and failed design attempts into evaluation and training assets. CADSmith's `data/dataset_v2/` (100 hand-written, hand-validated prompts) and `scripts/analyze_results.py` provide the initial corpus and analysis infrastructure.

2.2.7 Risk Containment

The system must gate manufacturing-sensitive decisions through human approval and policy checks. CADSmith has no approval gate mechanism; Temporal Signals provide this in the production platform.

2.2.8 Tool Portability

The system must keep agent semantics above any single editor, kernel, mesh library, or model provider. CADSmith's agents use Claude Sonnet for generation and Claude Opus for the Judge; the platform must not couple the semantic primitive layer to any specific model or version.

3. System Architecture

3.1 Architecture Principle

The architecture separates user-facing CAD editing from geometry authority. ForgeCAD is the presentation and editable handoff surface — not the geometry authority. CadQuery owns canonical solid generation (as in `executor.py`). MeshLib owns mesh inspection and repair. The Geometry Agent Runtime exposes semantic primitives above both.

3.2 Architecture Layers

Layer UX — ForgeCAD (Presentation and Editable Surface)

Requirement: ForgeCAD receives generated editable model code, previews, exports, and review artifacts. It must not act as geometry authority at any stage.

CADSmith baseline: CADSmith exports STEP and STL via `executor.py` but has no ForgeCAD integration. This layer is a new production addition.

Layer GRT — Geometry Agent Runtime (Semantic Primitive Layer)

Requirement: The runtime exposes typed primitives including at minimum: mounting plates, holes, ribs, clearances, mates, fillets, fixtures, and inspection operations. The runtime compiles those primitives into tool-specific operations and preserves a compact execution trace.

CADSmith baseline: `pipeline.py` is the direct orchestration baseline. The Planner → Coder → Executor → Validator → Refiner chain in `agents.py` maps onto the primitive planning and execution loop. The Error Refiner implements the inner execution-error repair loop (up to 3 retries); the Refiner implements the outer geometric-error repair loop (up to 5 iterations). Both loops must be preserved and made configurable in the production runtime.

Layer CAD — CadQuery + OCCT/MeshLib (Geometry Authority)

Requirement: CadQuery solids must be canonical during generation. The OCCT kernel used in `executor.py` provides exact geometric measurements (bounding box, volume, face counts, solid validity) and must be retained as the deterministic validation layer. MeshLib extends this with mesh inspection, repair, proximity checks, and watertightness testing. The system must never rely on screenshots or editor code as the source of truth — this constraint is validated by CADSmith's benchmark (§0.3).

CADSmith baseline: `executor.py` performs sandboxed subprocess execution of CadQuery code, extracts geometry from the OCCT kernel, and exports STEP/STL. This is the authoritative

geometry execution implementation. MeshLib is an additive capability beyond CADSmith's current OCCT scope.

Layer RLM — Recursive and Multimodal Reasoning

Requirement: RLM handles long-context exploration and decomposition. Multimodal calls must evaluate rendered views and raw geometry evidence. The reasoning layer must support focused sub-calls and context retention across multi-step geometry operations.

CADSmith baseline: Two RAG knowledge bases must be used and maintained:

- `rag_kb1.py` — 155 CadQuery API entries + 28 worked examples; used by Planner and Coder agents
- `rag_kb2.py` — 25 error-solution patterns for common CadQuery failures; used by Error Refiner and Refiner agents

Model assignment from CADSmith must be preserved: Claude Sonnet handles generation (Planner, Coder, Refiner); Claude Opus handles independent verification (Judge). This separation prevents self-confirmation bias in verification.

Layer WF — Temporal (Durable Stage Orchestration)

Requirement: Temporal must own workflow durability across coarse stages: planning, generation attempt, inspection, repair, verification, approval, and export. Primitive-level detail stays in trace artifacts.

CADSmith baseline: CADSmith has no durable orchestration layer. `pipeline.py` runs as a synchronous Python script with no crash recovery or approval gates. Temporal is the complete production addition required to make the CADSmith loop production-safe.

3.3 Data Flow Requirement

INTENT → DURABLE LOOP → 3D REASONING → GEOMETRY AUTHORITY → EDITABLE CAD + TRACE

Concretely:

1. Engineer submits prompt, rubric, and approval policy via the Product API
2. Product API starts a Temporal workflow
3. Planning Worker (CADSmith Planner + `rag_kb1.py`) emits a typed semantic primitive plan
4. Geometry Worker (CADSmith Coder + `executor.py`) runs CadQuery in a sandboxed subprocess, extracts OCCT measurements, invokes MeshLib
5. Verifier Worker (CADSmith Judge + `validator.py` + `render.py`) scores using geometry measurements and rendered views
6. On pass: ForgeCAD Adapter emits editable code, previews, STEP/STL, and trace artifacts
7. On fail: bounded repair (extending CADSmith inner/outer loops) or human clarification before re-entry

8. All outcomes persisted to Artifact Store; Temporal history holds references only

3.4 Artifact Store Requirement

The system must maintain a persistent Artifact Store holding: canonical solids, OCCT measurements, mesh evidence, rendered views, primitive traces, and verifier labels. CADSmith currently writes outputs to a local `outputs/` directory; the production artifact store replaces this with durable, referenced storage. Temporal workflow history must reference artifacts by ID only.

4. Spatial Reasoning Requirements — Thinking in 3D

4.1 Dual-Mode Verification Requirement

The harness must treat 3D reasoning as both an in-loop design capability and a post-generation verification capability:

- **In-loop:** Agent observes geometry evidence, revises primitives, and repairs defects during generation — implemented in CADSmith's outer refinement loop (`pipeline.py`)
- **Post-generation:** Verifier evaluates whether the final artifact satisfies intent using both rendered views and deterministic checks — implemented in CADSmith's `validator.py`

4.2 Semantic Primitive Planning

The agent must plan in typed CAD concepts. Minimum required vocabulary: holes, ribs, mounting plates, offsets, mates, clearances, fastener patterns, envelopes, and manufacturability constraints. CADSmith's Planner agent (`agents.py`) converts natural language into a structured design spec; the production runtime must formalize this into a validated typed schema, not a freeform spec string.

4.3 Raw Geometry Evidence Requirements

The system must expose deterministic geometry evidence. CADSmith's `executor.py` already extracts from the OCCT kernel: bounding box, volume, face counts, and solid validity. The production platform extends this with: topology, intersections, proximity, normals, watertightness, face references, and mesh defects from MeshLib. This evidence must be available as structured data, not only as rendered images.

4.4 Rendered Visual Evidence Requirements

The system must produce multi-view renders for multimodal inspection. CADSmith's `render.py` implements VTK-based three-view rendering:

- **Isometric view** — overall shape assessment
- **High-angle rear view** — reveals top-face features such as bolt holes and bores
- **Front profile** — shows vertical structure

These three views are the minimum required for the Verifier. They must be stored as canonical evidence artifacts in the Artifact Store. For complex parts (T3 tier), the known limitation is that fixed views can miss small geometric gaps (§0.3, T3_019 drone frame failure). Adaptive view selection must be planned for Phase 2.

4.5 Repair and Re-Verification Loop

Failures must route back into the primitive plan. CADSmith implements two nested repair loops that must be preserved and extended:

- **Inner loop** (`(pipeline.py)`): Handles execution errors (bad CadQuery code → Error Refiner agent → retry using `(rag_kb2.py)` context). Maximum depth: 3 attempts.
- **Outer loop** (`(pipeline.py)`): Handles geometric errors (wrong shape → Validator feedback → Refiner → re-execute). Maximum depth: 5 iterations.

Both loop caps must be configurable per workflow. Exceeding either cap without resolution must trigger human escalation, not silent failure.

4.6 Why Visual-Only Verification Is Prohibited

A rendered screenshot cannot prove constraints, tolerances, clearances, topology, or manufacturability. CADSmith quantifies this: removing vision from T3 parts raises mean Chamfer Distance from 1.42 to 49.68. The system must combine visual inspection (`(render.py)` + `(validator.py)`) with geometric measurements (`(executor.py)` OCCT extraction). Verification based solely on rendered images is not compliant.

4.7 Independent Judge Requirement

The verification Judge must use a model independent from the code-generation agents. CADSmith implements this by assigning Claude Opus to the Judge while Claude Sonnet handles Planner, Coder, Error Refiner, and Refiner roles. This separation prevents the system from grading its own homework. This model-role separation is a required architectural constraint.

5. Canonical State Ownership

The architecture assigns one clear owner for each representation. Treating generated editor code, preview images, meshes, and solids as interchangeable sources of truth is explicitly prohibited.

5.1 Ownership Table

Representation	Canonical Owner	CADSmith Source	Role in the Harness
Design Intent	Workflow input + user approval state	<code>run.py</code> prompt input	Stable requirements, constraints, review decisions, manufacturing caveats
Semantic Plan	Geometry Agent Runtime	<code>agents.py</code> Planner output	Typed primitive sequence, auditable and portable across tools
Generated Solids	CadQuery via OCCT	<code>executor.py</code>	Canonical during parametric construction and feature generation
Mesh Evidence	MeshLib + OCCT kernel	<code>executor.py</code> bounding box, volume, validity	Canonical for mesh inspection, repair, collision, proximity, watertightness
Rendered Views	Geometry Runtime artifact store	<code>render.py</code> VTK three-view output	Evidence for multimodal review, human approval, regression comparison
Editable CAD	ForgeCAD	New addition (CADSmith exports STEP/STL only)	User-facing code, preview surface, handoff artifact; not geometry authority
Durable State	Temporal	New addition (not in CADSmith)	Coarse stage transitions, retries, approval waits, signals, artifact references
Primitive Trace	Trace artifact store	<code>pipeline.py</code> loop state + <code>metrics.py</code> results	Compact execution detail for audits, evals, debugging, future fine-tuning

5.2 Ownership Violation Prohibition

- The system must never use ForgeCAD outputs as geometry validation evidence
- The system must never use rendered images as the sole proof of dimensional correctness (validated by CADSmith T3 vision-ablation data)
- The system must never treat Temporal workflow history as the geometry trace store

6. Geometry Agent Runtime Contract

6.1 Runtime Role

The Geometry Agent Runtime is the execution boundary between model reasoning and CAD tooling. The agent does not freestyle arbitrary CAD code as the primary interface. It proposes and revises semantic primitives; the runtime compiles those into CadQuery generation (via `executor.py`), MeshLib inspection and repair, render artifacts (via `render.py`), and ForgeCAD handoff code.

6.2 Required Runtime Primitives

Primitive	Function	CADSmith Baseline	Production Backing
<code>primitive_plan</code>	Create a typed plan of CAD features, constraints, dimensions, and expected evidence	<code>agents.py</code> Planner + <code>rag_kb1.py</code> (155 API entries, 28 examples)	RLM + schema validation
<code>solid_generate</code>	Compile semantic primitives into canonical solids and parametric construction steps	<code>agents.py</code> Coder + <code>executor.py</code> sandboxed subprocess	CadQuery via OCCT
<code>mesh_inspect</code>	Check watertightness, topology, intersections, normals, clearances, and mesh defects	<code>executor.py</code> OCCT extraction (bounding box, volume, face count, validity)	MeshLib (extended beyond OCCT)
<code>mesh_repair</code>	Repair or simplify mesh outputs without hiding failed design intent	Not in CADSmith; outer loop revises source code rather than patching meshes	MeshLib + policy gates
<code>measure_geometry</code>	Collect dimensions, volumes, bounding boxes, face references, tolerance evidence	<code>executor.py</code> OCCT kernel extraction	CadQuery + MeshLib
<code>render_views</code>	Produce canonical visual evidence:	<code>render.py</code> VTK three-view (isometric, high-angle rear, front	Geometry renderer;

Primitive	Function	CADSmith Baseline	Production Backing
	isometric, high-angle rear, front profile, section, exploded	profile)	extend VTK views for production
<code>visual_verify</code>	Use multimodal models to compare rendered evidence against user intent and rubric	<code>validator.py</code> LLM-as-Judge (Claude Opus) cross-referencing prompt, code, kernel metrics, and three-view render	Thinking with Images / 3D verifier; must remain model-independent
<code>forgecad_emit</code>	Generate editable ForgeCAD model code and attach previews, exports, and review notes	No direct analog; STEP/STL export from <code>executor.py</code> is the closest foundation	ForgeCAD adapter (new)
<code>trace_capture</code>	Persist primitive plan, tool evidence, verification scores, and repair attempts as artifacts	<code>scripts/analyze_results.py</code> + <code>metrics.py</code> result bundles	Artifact store + Temporal reference
<code>approval_gate</code>	Block release or export when manufacturing-sensitive assumptions need human signoff	Not present in CADSmith	Temporal Signal (new)

6.3 Primitive Schema Validation Requirement

Every `primitive_plan` output must be validated against a typed schema before any geometry tool execution is triggered. CADSmith's Planner emits an unstructured design spec string; the production runtime must formalize this into a schema with explicit field types, required fields, and unsupported-feature errors. Schema failures must produce an explicit error, not degraded execution.

6.4 `mesh_repair` Policy Constraint

`mesh_repair` must not hide design mistakes. CADSmith does not implement mesh repair; the outer refinement loop revises source code rather than patching meshes. The production `mesh_repair` primitive must be designed to avoid silently papering over design intent failures.

Every repair action must be recorded in the trace. Repair attempts must be capped. Non-trivial geometry changes must require human review.

6.5 RAG Knowledge Base Maintenance

Both CADSmith knowledge bases are living assets that must be versioned and maintained:

- `rag_kb1.py` (155 CadQuery API entries + 28 worked examples) — extend as new primitive types are added and as CadQuery API coverage grows
- `rag_kb2.py` (25 error-solution patterns) — update as new failure modes are discovered through the trace flywheel and failure taxonomy

Breaking changes to either knowledge base must trigger regression evaluation against `data/dataset_v2/`.

7. Minimum Viable Workflow

7.1 MVP Scope

The first implementation must prove the full spatial reasoning loop before expanding into multi-part child workflows or fine-tuning. The MVP must demonstrate in a single cohesive loop: language-to-geometry, deterministic inspection, visual reasoning, editable output, and human approval.

7.2 MVP Workflow Steps

Step 1 — Prompt and Requirements Ingestion

User intent enters as a workflow input. The system must extract: dimensions, constraints, assumptions, manufacturing risk, and required review evidence. CADSmith's `run.py` provides the minimal prompt entry model; the production platform extends this with project context, output target, and evaluation rubric as structured input fields. Ambiguous intent must be flagged before proceeding to planning.

Step 2 — Semantic Primitive Plan

The Planning Worker (derived from CADSmith's Planner agent + `rag_kb1.py`) creates a typed plan using feature and inspection primitives. The plan must be validated against the primitive schema before any geometry tool executes. CADSmith emits an unstructured spec; the production system must enforce typed schema validation as the gate between planning and execution.

Step 3 — CadQuery Solid Generation

The Geometry Worker (derived from CADSmith's Coder agent + `executor.py`) compiles primitives into canonical CadQuery solids in a sandboxed subprocess, retaining parameters and construction trace. If execution fails, the inner repair loop activates: Error Refiner agent, up to 3

retries, using `rag_kb2.py` error-solution patterns as context. The solid generation step must produce a parametric construction history.

Step 4 — OCCT/MeshLib Inspection and Repair

The Geometry Worker extracts OCCT kernel measurements from `executor.py` (bounding box, volume, face counts, solid validity) and invokes MeshLib for extended mesh inspection. Repair attempts must be bounded and recorded. The repair cap must not be exceeded without human escalation.

Step 5 — Multi-View 3D Verification

The Verifier Worker (derived from CADSmith's Judge agent + `validator.py` + `render.py`) renders three views (isometric, high-angle rear, front profile) and scores against intent using raw geometry evidence plus rendered views. The Judge must use a model independent from generation agents (Claude Opus as in CADSmith). Failures must return to the primitive plan with an explicit failure reason and route through the outer repair loop (Refiner agent, up to 5 iterations) before human escalation.

Step 6 — ForgeCAD Handoff and Approval

The system emits editable ForgeCAD code, previews, STEP/STL exports (extending `executor.py` export), and trace artifacts. Human approval gates (Temporal Signal) must be in place before releasing any manufacturing-sensitive output for export. This step has no direct CADSmith analog; it is the production addition that makes agent output actionable by engineers.

7.3 Multi-Part Deferral Requirement

Multi-part orchestration, model fine-tuning, and broad export coverage must be deferred until the single-part Thinking in 3D loop passes the decision metrics in Section 11.

8. Worker Boundaries and Execution Surfaces

8.1 Ownership Model

Temporal owns durable progression and retries. Workers own bounded execution. The Artifact Store owns heavy geometry evidence. ForgeCAD owns the editable human handoff. These boundaries must not be blurred.

8.2 Worker Definitions

Worker W-01 — Design Workflow (Durable)

- **Queue:** `design`
- **Owns:** Stage order, retries, timeouts, signals, queries, and artifact references; does not own primitive execution
- **CADSmith analog:** Top-level `pipeline.py` run loop; the production worker wraps this in Temporal for crash recovery and approval gating

- **Failure mode:** Retry with backoff or gate for human intervention

Worker W-02 — Planning Worker (Reasoning)

- **Queue:** `planning`
- **Output:** Typed primitive plan plus assumptions; may ask for clarification when intent is underspecified
- **CADSmith analog:** Planner agent in `agents.py`, augmented by `rag_kb1.py` (155 CadQuery API entries + 28 worked examples)
- **Failure mode:** Ambiguity escalation to requester

Worker W-03 — Geometry Worker (Geometry)

- **Queue:** `geometry`
- **Execution environment:** Isolated sandboxed subprocess (as in `executor.py`)
- **Produces:** Canonical CadQuery solids, OCCT kernel measurements, MeshLib diagnostics, STEP/STL exports
- **CADSmith analog:** Coder agent + `executor.py`; inner repair loop via Error Refiner agent + `rag_kb2.py` (25 error-solution patterns); max 3 retries
- **State:** Artifact trace persisted to store
- **Failure mode:** Invalid geometry with explicit error and trace

Worker W-04 — Verifier Worker (Evidence)

- **Queue:** `verify`
- **Input:** Rendered images from `render.py` VTK three-view + raw OCCT geometry measurements
- **Decision:** Pass, bounded repair (max 5 outer loop iterations via Refiner agent), or escalation to human review
- **CADSmith analog:** Judge agent (Claude Opus) in `agents.py` + `validator.py` + `render.py`; must remain model-independent in production
- **Failure mode:** Mismatch with failure category, evidence bundle, and render artifact

Worker W-05 — ForgeCAD Adapter (Handoff)

- **Queue:** `handoff`
- **Condition:** Executes only after accepted geometry has passed the verifier or approval gate
- **Output:** Editable ForgeCAD model code, previews, STEP/STL (from `executor.py` export), and review notes
- **CADSmith analog:** No direct analog; STEP/STL export in `executor.py` is the closest foundation
- **Failure mode:** Translation drift with explicit diagnostic

8.3 Failed Verification Re-Entry Rule

Failed verification must re-enter as a new planning attempt. It must not be handled as hidden state mutation. CADSmith's outer loop routes Validator feedback to the Refiner agent and re-executes; the production system must make this a first-class Temporal workflow stage, not a synchronous loop embedded in a single activity.

8.4 Sandbox and Isolation Requirement

CADSmith's `executor.py` already runs CadQuery in a sandboxed subprocess. The production platform extends this with: resource limits enforced, read-only inputs where possible, no ambient credentials, timeout enforcement, and artifact size quotas.

9. State Management and Durability

9.1 Temporal State Scope Requirement

Temporal must record coarse workflow stages, not every semantic primitive call or CadQuery generation attempt. Workflow history must capture: stage boundaries, activity outcomes, artifact references, retries, timeouts, signals, queries, and approval decisions.

9.2 Trace Artifact Scope Requirement

The Geometry Runtime must record primitive calls, parameters, measurements, renders, mesh diagnostics, repair attempts, and verifier scores as trace artifacts external to Temporal.

CADSmith's `scripts/analyze_results.py` and `metrics.py` result bundles prototype what a trace artifact contains; the production trace schema must formalize and extend these fields.

9.3 Temporal Query Requirements

Temporal Queries must expose without mutating workflow state:

- Current phase
- Latest preview artifact URI
- Verification score (Chamfer Distance, F1, IoU from `metrics.py` model)
- Failure reason and failure category
- Artifact URIs (solid, mesh, render, trace)
- Approval requirement status

9.4 Temporal Signal Requirements

Signals must support into a running workflow: approval, rejection, parameter changes, rubric updates, and iteration requests — without polling or restarting.

9.5 History Management Rule

Primitive traces must be attached to workflow history by reference, not embedded in full.

Continue-As-New must trigger before 40,000 events. CADSmith's synchronous `pipeline.py` has no equivalent constraint; the production Temporal wrapper must enforce this from day one.

10. Product API Contract

10.1 API Design Principle

The external API wraps Temporal workflow operations and artifact access. Starting a design creates a workflow; iteration and approval become signals; status and previews are queries; large geometry evidence (including `metrics.py` outputs) is read from the artifact store by reference.

10.2 Required Endpoints

Method	Path	Description
POST	/api/v1/designs	Start a workflow from prompt, project context, output target, and evaluation rubric
SIGNAL	/api/v1/designs/:id/iterate	Send revision instructions, updated constraints, or changed review criteria
QUERY	/api/v1/designs/:id/status	Read phase, progress, verifier score (CD/F1/IoU), failure reason, and approval requirement
QUERY	/api/v1/designs/:id/code	Read current editable ForgeCAD source code and generation metadata
QUERY	/api/v1/designs/:id/evidence	Read OCCT kernel measurements, MeshLib diagnostics, three-view render bundle, and trace links
SIGNAL	/api/v1/designs/:id/approve	Approve, reject, or request changes at a human-review gate
SIGNAL	/api/v1/designs/:id/params	Send parameter changes into a running workflow for repair or regeneration
GET	/api/v1/designs/:id/trace	Fetch the primitive trace artifact and related Temporal history references
POST	/api/v1/designs/:id/export	Trigger accepted output export: ForgeCAD, STEP, STL (extending <code>executor.py</code>), mesh, preview pack, or report
WS	/ws/v1/designs/:id/stream	Stream stage changes, previews, verifier updates, approval gates, and export status

11. Performance Requirements and Decision Metrics

11.1 Metric Philosophy

These metrics are review gates, not marketing claims. All metrics must be measured in absolute millimeter space with ICP alignment, using the same methodology as CADSmith's `metrics.py`.

11.2 Decision Metric Table

Metric	Target	CADSmith Baseline	Interpretation
Valid single-part first pass rate	> 80%	100% exec (full pipeline with vision)	Measure over <code>data/dataset_v2/</code> T1 and T2 tiers
Average repair iterations per successful design	< 2	Outer loop cap: 5; inner loop cap: 3	High values indicate primitive weakness or RAG knowledge gap
Outputs with trace artifacts	100%	<code>scripts/analyze_results.py</code> captures all runs	Every execution, success or failure, must produce a trace
Silent geometry failures	0	<code>validator.py</code> always returns explicit pass/fail with evidence	All failures must be named, categorized, and surfaced
MVP single-part workflow completion time	< 5 minutes	Not benchmarked in CADSmith	End-to-end: prompt submission to ForgeCAD code emission
Temporal events before Continue-As-New	< 40,000	N/A (no Temporal in CADSmith)	History management bound
Human gates for risky exports	100%	Not present in CADSmith	No manufacturing-sensitive export bypasses approval

Metric	Target	CADSmith Baseline	Interpretation
Model and runtime comparison	A/B harness operational	<code>scripts/run_zeroshot_baseline.py</code> vs full pipeline	Must support A/B comparison between models and runtime versions

11.3 Geometry Quality Metrics

The platform must compute and expose from `metrics.py` for every completed attempt:

- **Chamfer Distance (CD)** — median and mean, absolute mm with ICP alignment. CADSmith full-pipeline-with-vision target: mean 0.74.
- **F1 Score** — surface coverage precision/recall. CADSmith target: 0.9846.
- **Volumetric IoU** — intersection-over-union of solid volumes. CADSmith target: 0.9629.

The production platform must meet or exceed these values on equivalent prompts from `data/dataset_v2/`.

12. Implementation Roadmap

12.1 Roadmap Principle

Phase 1 begins from the CADSmith codebase as the working executable baseline. The goal is to wrap and extend, not rebuild.

12.2 Phase 1 — MVP Geometry Loop (Weeks 1-4)

Goal: Wrap CADSmith's pipeline in Temporal and prove a durable single-part geometry loop.

Required deliverables:

- Implement Temporal workflow with coarse stages wrapping `pipeline.py` loop
- Define typed semantic primitive schema (formalizing CADSmith Planner spec output)
- Compile primitives into CadQuery solids via `executor.py` sandboxed subprocess
- Run OCCT kernel inspection from `executor.py` (bounding box, volume, face count, validity)
- Generate VTK three-view renders via `render.py` and persist as artifacts
- Run LLM-as-Judge verification via `validator.py` with Claude Opus as independent model
- Emit editable ForgeCAD code for accepted outputs

Phase gate: Engineering leads must submit a T1 or T2 prompt from `data/dataset_v2/` and receive editable ForgeCAD output with a trace artifact and OCCT measurements.

12.3 Phase 2 — Reasoning and Review (Weeks 5-8)

Goal: Add full RLM reasoning, MeshLib, approval gates, and evaluation harness.

Required deliverables:

- Add RLM context exploration using `rag_kb1.py` and `rag_kb2.py`
- Build 3D verifier extending `validator.py` with MeshLib evidence and geometry measurements
- Formalize inner (3-retry) and outer (5-iteration) repair loops as first-class Temporal stages
- Add Temporal Signal approval gates and Query status endpoints
- Persist primitive traces and verifier rubrics using `metrics.py` schema
- Create eval suite from `data/dataset_v2/` T1/T2/T3 tiers with A/B comparison against `run_zeroshot_baseline.py`
- Extend `rag_kb2.py` with new failure patterns discovered in Phase 1

Phase gate: Full Thinking in 3D loop must pass T1 and T2 decision metrics. CADSmith baseline (CD mean 0.74, F1 0.9846, IoU 0.9629) must be matched or exceeded.

12.4 Phase 3 — Scale and Productization (Weeks 9-12)

Goal: Harden the platform for concurrent use and activate the trace flywheel.

Required deliverables:

- Add multi-part child workflows after MVP loop is proven
- Introduce dedicated queues: planning, geometry, render, export
- Harden sandboxing extending `executor.py` subprocess model with resource limits, no ambient credentials, and artifact size quotas
- Add OpenTelemetry and workflow cost reporting
- Use trace corpus to extend `rag_kb1.py`, `rag_kb2.py`, and the primitive schema
- Enable distillation and fine-tuning from trace corpus after label quality stabilizes (seed: `data/dataset_v2/`)
- Load test concurrent workflows and large artifact payloads
- Validate T3 complex-part performance: must match CADSmith T3 baseline (mean CD 1.42 with vision vs 49.68 without)

Phase gate: Load testing must pass. T3 metrics must be validated with vision.

12.5 Deferral Policy

Multi-part orchestration, fine-tuning, and broad export support must be deferred until the single-part loop passes Phase 2 gate metrics. No Phase 3 work may begin while Phase 1 metrics are unmet.

13. Risks and Mitigations

13.1 Risk Register

Risk	Severity	Mitigation
Semantic primitives are too weak	High	Start with a narrow single-part primitive set seeded from <code>data/dataset_v2/</code> T1 prompts. Require schema, validation, examples, and explicit unsupported-feature errors before expanding scope.
Visual verifier misses geometric failures	High	Never rely on rendered images alone. Pair every <code>render.py</code> three-view with OCCT/MeshLib evidence. CADSmith's T3_019 drone frame (§0.3) is the benchmark for this risk; it must be included in the regression suite.
CadQuery to ForgeCAD translation drift	High	Treat CadQuery (via <code>executor.py</code>) as canonical during generation. Add translation tests comparing rendered and measured ForgeCAD outputs against the accepted STEP artifact from <code>executor.py</code> .
Mesh repair hides design mistakes	Medium	Separate repairable mesh defects from semantic design failures. Record every repair, cap attempts, require human review for non-trivial geometry changes. CADSmith has no mesh repair; introduce it carefully to avoid masking intent failures.
Temporal event history overflow	Medium	Persist primitive detail as artifacts and store references in history. Continue-As-New before 40,000 events. CADSmith's synchronous <code>pipeline.py</code> has no equivalent constraint.
RLM cost and latency amplification	Medium	Cap recursion depth, enforce inner loop (3) and outer loop (5) bounds from CADSmith, cache <code>rag_kb1.py</code> and <code>rag_kb2.py</code> context, route simple inspections to Sonnet-class models, reserve Opus for Judge verification.
Sandbox and tool execution risk	Medium	Extend CADSmith's <code>executor.py</code> subprocess model with resource limits, read-only inputs, no ambient credentials, timeout enforcement, and artifact size quotas.
Scope expands before MVP proof	Low	Defer multi-part orchestration, fine-tuning, and broad export until the single-part loop passes Phase 2 gate metrics against <code>data/dataset_v2/</code> T1 and T2 tiers.

13.2 Known Failure Mode — Fixed-View Blind Spot

CADSmith's benchmark documents a specific failure class (T3_019, quadcopter frame) that passed all validation checks (F1 = 0.963, IoU = 0.985) but contained small arm-hub gaps invisible

to fixed three-view rendering. This establishes the known ceiling of the `render.py` approach. The production verifier must acknowledge this ceiling and plan adaptive view selection or higher-resolution crops for Phase 2 or Phase 3. This failure type must be added to the failure taxonomy as: **"Near-miss geometric gap — fixed-view blind spot."**

13.3 High-Severity Risk Policy

All three High severity risks must have a mitigation owner assigned before development begins. Phase 1 progress is gated on mitigations being in place, not risks being resolved.

14. Trace Capture and Evaluation Pipeline

14.1 Trace-First Principle

Every attempt must become evaluation data before it becomes training data. CADSmith's `data/dataset_v2/` provides 100 hand-written, hand-validated prompts as the initial evaluation corpus — every reference script was written and inspected by humans, not LLM-generated, making it a trustworthy regression baseline. The trace corpus must first power regression tests and model/runtime comparisons; fine-tuning comes after label quality is trusted.

14.2 Required Trace Data Categories

Planning Traces

Must include: prompt interpretation, extracted constraints, assumptions, primitive plans (extending Planner agent output from `agents.py`), repair hypotheses, and unsupported-feature decisions.

Geometry Evidence

Must include: CadQuery construction metadata, OCCT kernel diagnostics from `executor.py` (bounding box, volume, face count, validity), MeshLib diagnostics, measurements, collision checks, clearances, repair outcomes, and artifact links.

Visual Evidence

Must include: canonical three-view render sets from `render.py`, annotated failure views, verifier prompts, Judge model judgments from `validator.py`, human review notes, and screenshot comparisons.

Outcome Labels

Must include: pass/fail reason, first-pass status, inner loop iteration count, outer loop iteration count, Chamfer Distance, F1, IoU (from `metrics.py`), human approval decision, export readiness, and regression category.

14.3 Trace-to-Evaluation Pipeline

Stage	Process	CADSmith Baseline	Output
1. Capture	Persist stage metadata, primitive trace, geometry evidence, rendered views, verifier scores, and human decisions after each attempt	<code>pipeline.py</code> loop state + <code>metrics.py</code> result bundle	Raw trace bundle
2. Normalize	Convert traces into stable schema with prompt, plan, artifacts, measurements, rendered evidence, repair actions, and outcome labels	<code>scripts/analyze_results.py</code> result schema	Queryable corpus
3. Classify	Tag failures by root cause using the seven-category taxonomy	CADSmith failure categories inferred from benchmark + T3_019 failure	Failure taxonomy
4. Regress	Replay <code>data/dataset_v2/</code> prompts against new models, schemas, rubrics, and runtime versions	<code>scripts/run_custom_benchmark.py</code> + <code>scripts/run_zeroshot_baseline.py</code>	Agent eval suite
5. Improve	Use trace patterns to add primitives, extend <code>rag_kb1.py</code> and <code>rag_kb2.py</code> , tighten rubrics, improve repair policies	Knowledge base versioning	Runtime and model backlog
6. Fine-Tune (deferred)	Only after label quality stabilizes, transform successful and failed traces into supervised or preference data	<code>data/dataset_v2/</code> as seed dataset	Training-ready dataset

14.4 Seven Failure Categories

The failure taxonomy must include exactly these seven categories:

1. Primitive gap
2. Geometry invalidity
3. Visual mismatch

4. Translation drift
5. Verifier miss
6. User ambiguity
7. **Near-miss geometric gap — fixed-view blind spot** (from CADSmith T3_019 drone frame failure)

14.5 Benchmark Corpus Integration

The `data/dataset_v2/` corpus must be used as the primary regression set:

- **T1 — Primitives** (50 entries): boxes, cylinders, cones, tori; 1–3 ops. Primary coverage test for basic primitive schema.
- **T2 — Engineering Parts** (25 entries): brackets, flanges, gears, plates with hole patterns; 3–8 ops. Primary coverage test for the MVP loop.
- **T3 — Complex Parts** (25 entries): lofts, sweeps, shells, revolves, multi-body unions; 5–15 ops. Primary test for vision-augmented verification and the outer repair loop.

Phase 2 gate requires T1 and T2 metrics to match or exceed the CADSmith full-pipeline-with-vision baseline. Phase 3 gate requires T3 metrics to match or exceed that baseline.

14.6 Model Tiering Strategy

Once the evaluation harness is stable, model routing must become cost-aware. Claude Sonnet handles routine planning (Planner, Coder, Refiner, Error Refiner as in CADSmith). Claude Opus is reserved for the independent Judge and for ambiguous intent, novel geometry, and high-risk verification. Routing policy must be driven by measured quality from the eval harness, not assumption.

14.7 Fine-Tuning Gate

Fine-tuning is deferred until:

1. Label quality is trusted by the team
2. The seven-category failure taxonomy is stable
3. The eval harness supports A/B comparison using `scripts/run_custom_benchmark.py` and `scripts/run_zeroshot_baseline.py`

CADSmith's `data/dataset_v2/` may serve as seed data for supervised fine-tuning after this gate is passed. Commencing fine-tuning before these conditions are met risks training on noisy geometry mistakes.

15. Decision Summary

15.1 Recommended Direction

Approve the Geometry Agent Harness as a platform capability for CAD agent evaluation and development. The architecture is strongest when ForgeCAD is treated as the editable product surface, not the geometry authority. CadQuery (via `executor.py`) owns solid generation. The OCCT kernel and MeshLib own inspection and repair evidence. The Geometry Agent Runtime exposes semantic primitives above both.

15.2 CADSmith as the Production Starting Point

The cloned CADSmith repository is not merely a reference — it is the executable Phase 1 baseline. The `pipeline.py` orchestration, `agents.py` agent chain, `executor.py` sandboxed execution, `validator.py` Judge, `render.py` three-view rendering, `rag_kb1.py` and `rag_kb2.py` knowledge bases, and `data/dataset_v2/` benchmark corpus must all be integrated, not rebuilt. Phase 1 work is: wrapping this pipeline in Temporal, formalizing the primitive schema, adding ForgeCAD handoff, and introducing approval gates.

15.3 Temporal Role Clarification

Temporal's role is durable orchestration across coarse stages. CADSmith's synchronous `pipeline.py` is the proof of concept; Temporal is what makes it production-safe. The runtime trace (extending `metrics.py` and `analyze_results.py`) is the detailed evidence layer for engineering review, evals, and eventual model improvement.

15.4 Next Decision Point

The next decision must be an MVP build focused on one thing: proving the Thinking in 3D loop for single-part CAD workflows against T1 and T2 prompts from `data/dataset_v2/`. The following must wait:

- Multi-part orchestration
- Model fine-tuning (even with `data/dataset_v2/` as seed data)
- Broad export coverage beyond STEP/STL

15.5 Architecture Non-Negotiables

1. CadQuery via OCCT (`executor.py`) is canonical during solid generation — no exceptions
2. OCCT kernel measurements and MeshLib are canonical for geometry inspection — no exceptions
3. ForgeCAD is a presentation surface, not a geometry authority — no exceptions
4. The verification Judge must use a model independent from generation agents (Claude Opus separate from Claude Sonnet, as in `agents.py`) — no exceptions
5. Zero silent failures — all failures must be named and categorized against the seven-category taxonomy

6. 100% trace coverage — every execution produces a trace artifact with CD, F1, and IoU from `metrics.py`
 7. 100% human gates for risky exports — no manufacturing-sensitive output bypasses approval
-

Engineering Design Review — Geometry Agent Harness · RLM + Temporal + CadQuery + MeshLib + ForgeCAD · CADSmith (jabarkle/CADSmith) · May 2026